



INSTITUTO FEDERAL

Mato Grosso do Sul

DESENVOLVIMENTO WEB 1:
CSS Animações

INTRODUÇÃO

As animações têm o mesmo efeito que as transições, mas podemos apontar aqui suas particularidades:

- **Gatilho** precisamos fazer algum evento para iniciar as transições, como passar o cursor do mouse sobre o elemento. As animações podem ser iniciadas sem nenhum gatilho.
- **Repetição** Podemos fazer com que uma animação se repita ou até mesmo definir um looping infinito, diferente das transições que acabam assim que a propriedade chegar ao valor final.
- **Quadros de Animação (keyframes)** as transições apenas fazem uma propriedade ir de um valor X para um valor Y. Nas animações podemos definir vários valores intermediários

- A propriedade **transition** trabalha de forma muito **simples** e **inflexível**. Você praticamente diz para o browser qual o **valor inicial** e o **valor final** para que ele aplique a transição **automaticamente**, controlamos praticamente apenas o **tempo de execução**.
- Para termos mais controle sobre a animação temos a propriedade **animation** que trabalha juntamente com a rule **keyframe**.

- Diferente das transições, as animações não são declaradas dentro do seletor de algum elemento. As animações são independentes.
- Para isso, definimos o `@keyframe` e damos um nome à nossa animação.

TRANSFORMAÇÕES

- Dentro do keyframe declarado nós precisamos declarar o início e o fim da animação. Para isso utilizamos o "from" e "to".
- Dentro de "from" indicamos os valores iniciais das propriedades que serão animadas.
- Dentro de "to" indicamos os valores finais das propriedades que serão animadas.

TRANSFORMAÇÕES

Nome da animação

Início da animação

```
16  
17 ▾ @keyframes rodar{  
18   from{  
20     transform: rotate(0deg);  
21   }  
22   to{  
23     transform: rotate(360deg);  
24   }  
}
```

Tipo da transformação

Graus de rotação

Fim da animação

PROPRIEDADE TRANSITION

animation

permite definir a transição entre dois estados de um elemento, é uma simplificação de oito propriedades que podemos declarar separadamente:

- **animation-name:** nome da animação;
- **animation-duration:** tempo de duração da animação;
- **animation-timing-function:** função da curva de velocidade;
- **animation-delay:** atraso da animação iniciar;
- **animation-iteration-count:** quantidade de vezes que a animação irá ocorrer;
- **animation-direction:** direção da animação;
- **animation-play-state:** indica se a animação está pausada ou executando;
- **animation-fill-mode:** indica como o elemento deve ficar quando a animação não estiver sendo executada.

PROPRIEDADE ANIMATION

animation: [nome] [duração] [aceleração] [atraso] [iteração] [direção];

EXEMPLO DE UTILIZAÇÃO

```
div{  
  animation: minhaanimacao 2s ease  
  0.5s infinite alternate;  
}
```

EXEMPLO DE UTILIZAÇÃO

```
div{  
  animation-name: minhaanimacao;  
  animation-duration: 2s;  
  animation-timing-function: ease;  
  animation-delay: 0.5s;  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
}
```

Veja mais em [MDN webdocs](#), [w3schools](#), [cssreference.io](#)

TRANSFORMAÇÕES

Propriedade	Definição
animation-name	Especificamos o nome da função de animação
animation-duration	Define a duração da animação. O valor é declarado em segundos.
animation-direction	Define se a animação irá acontecer ou não no sentido inverso em ciclos alternados. Ou seja, se a animação está acontecendo no sentido horário, ao acabar a animação, o browser faz a mesma animação no elemento, mas no sentido anti-horário. Os valores são alternate ou normal.
animation-iteration-count	Define o número de vezes que o ciclo deve acontecer. O padrão é um, ou seja, a animação acontece uma vez e pára. Pode ser também infinito, definindo o valor infinite no valor.

TRANSFORMAÇÕES

Propriedade	Definição
animation-timing-function	Descreve qual será a progressão da animação a cada ciclo de duração. Existem uma série de valores possíveis e que pode ser que o seu navegador ainda não suporte, mas são eles: ease, linear, ease-in, ease-out, ease-in-out, cubic-bezier(<number>, <number>, <number>, <number>) [, ease, linear, ease-in, ease-out, ease-in-out, cubic-bezier(<number>, <number>, <number>, <number>)]* O valor padrão é ease
animation-iteration-count	Define o número de vezes que o ciclo deve acontecer. O padrão é um, ou seja, a animação acontece uma vez e pára. Pode ser também infinito, definindo o valor infinite no valor.

ATIVIDADE PRÁTICA

- Vamos fazer girar? Primeiro você define a função de animação, no caso é o nosso código que define um valor inicial de 0 graus e um valor final de 360 graus. Agora vamos definir as características deste DIV.

EXEMPLO-ANIMACAO.HTML

```
<head>
  <link rel="stylesheet"
href="estilo.css">
</head>
<body>
  <div></div>
</body>
```

ESTILO.CSS

```
.quadrado{
  width: 33px;
  height: 33px;
  background: black;
}
```

TRANSFORMAÇÕES

acrescente ao código, as propriedades de animation-name, animation-duration, animation-timing-function, animation-iteration-count e animation-direction:

ESTILO.CSS

```
div{  
    width: 100px;  
    height: 100px;  
    margin: 10% auto;  
    background: black;  
    animation-name: rodar;  
    animation-duration: 5s;  
    animation-timing-function: linear;  
    animation-iteration-count: black;  
    animation-direction: alternate;  
}
```

Animações com várias mudanças

- Vimos que dentro da declaração de uma animação declaramos apenas um valor inicial e um valor final para a propriedade animada. Mas uma grande diferença das animações em relação às transições é a possibilidade de fazer várias alterações intermediárias.
- Para isso utilizamos porcentagem, que indicará em que momento da animação teremos a alteração.

Animações com várias mudanças

declaramos nossa animação definindo os estados inicial e final com `from` e `to`, podemos ter o mesmo efeito com as porcentagens:

ESTILO.CSS

```
@keyframes rodar{  
  from {  
    transform: rotate(0deg);  
  }  
  to {  
    transform: rotate(360deg);  
  }  
}
```

ESTILO.CSS

```
@keyframes rodar{  
  0% {  
    transform: rotate(0deg);  
  }  
  100% {  
    transform: rotate(360deg);  
  }  
}
```

Animações com várias mudanças

E podemos colocar outros valores entre 0% e 100%, como:

ESTILO.CSS

```
@keyframes rodar{  
  0% {  
    transform: rotate(0deg);  
    margin-left: 0%;  
  }  
  50% {  
    transform: rotate(360deg);  
    margin-left: 95%;  
  }  
  100% {  
    transform: rotate(0deg);  
    margin-left: 0%;  
  }  
}
```

MATERIAIS COMPLEMENTARES

- <https://www.chiefdesign.com.br/css3-animation-tutorial/>
- <https://cssanimation.rocks/pt/transition-vs-animation/>
- [MDN - animações css](#)
- [como-criar-uma-animacao-css3-com-sprites-firefox-os](#)
- <https://blog.teamtreehouse.com/css-sprite-sheet-animations-steps>
- [linhadecodigo -](#)
html5-canvas-movendo-um-personagem-com-sprites.aspx
- [tutoriais com funcionalidades interessantes para o site](#)

CSS ANIMAÇÕES

- <http://bouncejs.com/>
- <https://www.minimamente.com/project/magic/>
- <http://cssanimate.com/>
- [medium - animações-com-wow-js-e-animate-css](http://medium.com/animacoes-com-wow-js-e-animate-css)
- <http://www.justinaguilar.com/animations/index.html>
- <http://anijs.github.io/>
- <http://jeremyckahn.github.io/styleie/>
- <http://maccman.github.io/gfx/>

- <https://html-css-js.com/css/generator/>
- <https://www.css3maker.com/>
- <https://webcode.tools/css-generator>
- <https://www.cssportal.com/css3-menu-generator/#>
- <https://www.cssmatic.com>
- [smashingmagazine - 50-ferramentas de css uteis](#)
- <https://cssreference.io/>
- <https://htmlreference.io/>

REFERÊNCIAS

- <https://htmlreference.io/>
- <https://cssreference.io>
- <https://css-tricks.com/snippets/css/>
- <https://tympanus.net/codrops/category/playground/>
- <https://www.awwwards.com/>
- <https://dribbble.com/>
- <https://www.typewolf.com/>